

TinyMPC + Differential Flatness: Complete Cheat Sheet (5-Variant Hierarchy)

Jianghan Zhang

February 2026

Abstract

TinyMPC solves linear MPC via ADMM with offline-cached Riccati matrices, achieving microcontroller-grade speed at the cost of a fixed linearisation point. We observe that for differentially flat systems the linearisation is a smooth function of the flat output and its derivatives—a low-dimensional parameter $\alpha \in \mathbb{R}^d$ (e.g. $d = 4$ for a quadrotor). This gives a family of controllers forming a **complexity cascade**:

Online Relin $\xrightarrow{\text{cache}}$ Flat Template $\xrightarrow{K \rightarrow 0}$ Barrier Templ. $\xrightarrow{\mu \rightarrow 0}$ Analytic Templ. $\xrightarrow{\text{1st order}}$ Gain Schedule

Moving down the cascade trades constraint fidelity for real-time guarantees, from $O(n^3)$ online NMPC to ~ 260 -operation deterministic feedback.

Contents

1	TinyMPC Formulation	3
1.1	Problem	3
1.2	ADMM solver	3
1.3	Limitation: fixed linearisation	4
2	Differential Flatness	4
2.1	Flat equilibrium manifold	4
3	Variant 1: Online Relinearization	5
4	Variant 2: Flat Template	5
4.1	Parameterised linearisation	5
4.2	Riccati cache family	5
4.3	The template	5
5	Flat Template Algorithm	6
6	Uniform Tunnel Guarantee	6
6.1	Tunnel in original coordinates	7

7	Variant 3: Analytic Template (Zero-Iteration MPC)	7
7.1	Key observation	7
7.2	Quadrotor: explicit $A(\alpha), B(\alpha)$	7
7.3	Perturbation expansion of K_∞	8
7.4	The zero-iteration control law	8
8	Variant 4: Barrier-Augmented Template	8
8.1	Interior-point formulation	9
8.2	Input barrier Hessian	9
8.3	State barrier Hessian	9
8.4	Barrier-augmented DARE	9
8.5	α -dependent conservatism	10
8.6	Optional: IPM refinement	10
8.7	Limitation: reference outside feasible region	10
9	Barrier-Augmented Algorithm	11
10	Foolproof Implementation	11
10.1	Constants (your quadrotor)	11
10.2	Offline (run once, on your laptop)	12
10.3	<code>linearise</code> (α)	13
10.4	Online (every control cycle)	14
11	FPGA Architecture	15
11.1	Pipeline stages	15
11.2	Resource estimate (quadrotor)	15
12	Complete Hierarchy	15
13	Experiment Results	16
13.1	Hover stabilization	16
13.2	Figure-eight tracking	16
13.3	Circle-in-square (constrained tracking)	16
14	Python API Reference	17
14.1	Source modules	17
14.2	Flat linearization API	17
14.3	Analytic Template API	18
14.4	Barrier Template API	18
14.5	Flat Template API	18
14.6	TinyMPC API	19
15	Discussion	19
15.1	Connection to TTT	19
15.2	Why Barrier Cannot Enforce Absolute Constraints	19
15.3	Gain scheduling, done right	19
15.4	Higher-order templates	19
15.5	Beyond quadrotors	20
15.6	What this does not do	20

1 TinyMPC Formulation

We follow the formulation of [1].

1.1 Problem

Given discrete-time linear dynamics

$$x_{k+1} = A x_k + B u_k, \quad x_k \in \mathbb{R}^n, \quad u_k \in \mathbb{R}^m, \quad (1)$$

TinyMPC solves the finite-horizon constrained LQR:

$$\begin{aligned} \min_{x,u} \quad & \sum_{k=0}^{N-1} \left(\frac{1}{2} x_k^\top Q x_k + \frac{1}{2} u_k^\top R u_k + q^\top x_k + r^\top u_k \right) + \frac{1}{2} x_N^\top Q_f x_N + q_f^\top x_N \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \quad k = 0, \dots, N-1, \\ & x_{\min} \leq x_k \leq x_{\max}, \\ & u_{\min} \leq u_k \leq u_{\max}, \end{aligned} \quad (2)$$

with $Q, Q_f \succeq 0, R \succ 0$.

1.2 ADMM solver

Introduce slack variables $v = (v_x, v_u)$ to decouple dynamics from box constraints. The augmented Lagrangian with penalty $\rho > 0$ is

$$\mathcal{L}_\rho = J(x, u) + \sum_k \left(\lambda_{x,k}^\top (x_k - v_{x,k}) + \lambda_{u,k}^\top (u_k - v_{u,k}) + \frac{\rho}{2} \|x_k - v_{x,k}\|^2 + \frac{\rho}{2} \|u_k - v_{u,k}\|^2 \right). \quad (3)$$

Each ADMM iteration:

- (i) **Primal update** (Riccati backward pass). Solve the unconstrained LQR with shifted cost:

$$\begin{aligned} P_N &= Q_f + \rho I, \\ K_k &= (R + \rho I + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A, \\ P_k &= Q + \rho I + A^\top P_{k+1} A - A^\top P_{k+1} B K_k. \end{aligned} \quad (4)$$

The gain K_k and the affine term $d_k = (R + \rho I + B^\top P_{k+1} B)^{-1} (B^\top p_{k+1} + r + \rho (v_{u,k} - \lambda_{u,k}/\rho))$ produce the primal trajectory via forward rollout: $u_k = -K_k x_k - d_k, x_{k+1} = A x_k + B u_k$.

- (ii) **Slack update** (element-wise projection):

$$v_{x,k} = \text{clip}(x_k + \lambda_{x,k}/\rho, x_{\min}, x_{\max}), \quad v_{u,k} = \text{clip}(u_k + \lambda_{u,k}/\rho, u_{\min}, u_{\max}). \quad (5)$$

- (iii) **Dual update**:

$$\lambda_{x,k} += \rho (x_k - v_{x,k}), \quad \lambda_{u,k} += \rho (u_k - v_{u,k}). \quad (6)$$

Remark 1.1 (Offline cache). When A, B, Q, R, ρ are fixed, the Riccati matrices $\{P_k, K_k\}$ converge to steady-state values P_∞, K_∞ as $N \rightarrow \infty$. TinyMPC precomputes and caches P_∞, K_∞ and the matrix $C_\infty := (R + \rho I + B^\top P_\infty B)^{-1}$ offline. Online cost: one forward pass (matrix-vector products) + two element-wise projections + one dual update. No matrix inversions at runtime.

1.3 Limitation: fixed linearisation

For a nonlinear system $x_{k+1} = f(x_k, u_k)$, TinyMPC uses a single linearisation

$$A = \left. \frac{\partial f}{\partial x} \right|_{(\bar{x}, \bar{u})}, \quad B = \left. \frac{\partial f}{\partial u} \right|_{(\bar{x}, \bar{u})}, \quad (7)$$

at a fixed nominal $(\bar{x}, \bar{u})$ (typically hover for a quadrotor). The cached matrices are valid only near $(\bar{x}, \bar{u})$. Deviation $\rightarrow$ model mismatch $\rightarrow$ degraded or lost feasibility.

2 Differential Flatness

Definition 2.1 (Differentially flat system). *A system $\dot{x} = f(x, u)$ with state $x \in \mathbb{R}^n$, input $u \in \mathbb{R}^m$ is **differentially flat** if there exist **flat outputs** $\sigma = h(x, u, \dot{u}, \dots, u^{(r)}) \in \mathbb{R}^m$ such that:*

- (a) $x = \varphi(\sigma, \dot{\sigma}, \dots, \sigma^{(s)})$ (state recovery),
- (b) $u = \psi(\sigma, \dot{\sigma}, \dots, \sigma^{(s+1)})$ (input recovery),

for smooth maps φ, ψ and some finite s .

Remark 2.2 (Quadrotor). *For a quadrotor with state $(p, v, R, \omega) \in \mathbb{R}^3 \times \mathbb{R}^3 \times \text{SO}(3) \times \mathbb{R}^3$ and inputs $(T, \tau) \in \mathbb{R} \times \mathbb{R}^3$ (thrust + body torques), the flat outputs are*

$$\sigma = (x, y, z, \psi) \in \mathbb{R}^3 \times S^1 \approx \mathbb{R}^4. \quad (8)$$

Recovery: position $p = \sigma_{1:3}$, velocity $v = \dot{\sigma}_{1:3}$, thrust direction $e_3^B = (m\ddot{\sigma}_{1:3} + mg e_3) / \|m\ddot{\sigma}_{1:3} + mg e_3\|$ (determines roll and pitch from $\ddot{\sigma}$), yaw from $\sigma_4 = \psi$. The full rotation R and angular velocity ω follow from $(\ddot{\sigma}, \ddot{\sigma}, \psi, \dot{\psi})$. Thrust $T = \|m\ddot{\sigma}_{1:3} + mg e_3\|$.

2.1 Flat equilibrium manifold

Given a flat output trajectory $\sigma(t)$ and its derivatives, the flatness maps produce a **dynamically consistent** pair:

$$x^*(\alpha) = \varphi(\alpha), \quad u^*(\alpha) = \psi(\alpha), \quad \alpha := (\sigma, \dot{\sigma}, \dots, \sigma^{(s+1)}). \quad (9)$$

This pair satisfies the dynamics exactly: $f(x^*(\alpha), u^*(\alpha)) = \dot{x}^*(\alpha)$.

Definition 2.3 (Flat parameter). *For a given flat system, the **flat parameter** is the minimal sub-tuple of α on which the linearisation (A, B) depends. Denote it $\alpha \in \mathbb{R}^d$.*

Remark 2.4 (Quadrotor flat parameter). *For a quadrotor, the Jacobians $\partial f / \partial x$ and $\partial f / \partial u$ evaluated at (x^*, u^*) depend only on:*

$$\alpha = (\ddot{\sigma}_{1:3}, \psi) = (a_x, a_y, a_z, \psi) \in \mathbb{R}^4, \quad (10)$$

where (a_x, a_y, a_z) is the commanded acceleration (determining the equilibrium attitude) and ψ is the yaw angle. At hover: $\alpha_0 = (0, 0, 0, 0)$.

3 Variant 1: Online Relinearization

The most accurate but most expensive variant. At each control step:

1. Compute α from the trajectory reference.
2. Evaluate $(A(\alpha), B(\alpha))$ via autograd or finite-difference Jacobians.
3. Solve a fresh DARE: $O(n^3)$.
4. Run K ADMM iterations with the new cache: $K \cdot O(n^2)$.

Online cost: $O(n^3 + K n^2)$ per step.

This is the “gold standard” that all other variants approximate. The per-step DARE solve is the bottleneck (~ 22 s total for 200 figure-eight steps in Python).

4 Variant 2: Flat Template

4.1 Parameterised linearisation

Linearise f at the flat equilibrium $(x^*(\alpha), u^*(\alpha))$:

$$A(\alpha) = \left. \frac{\partial f}{\partial x} \right|_{(x^*(\alpha), u^*(\alpha))}, \quad B(\alpha) = \left. \frac{\partial f}{\partial u} \right|_{(x^*(\alpha), u^*(\alpha))}. \quad (11)$$

Proposition 4.1 (Smoothness). *If f, φ, ψ are C^2 , then $\alpha \mapsto (A(\alpha), B(\alpha))$ is C^1 on $\mathbb{R}^d$.*

Proof. Composition of C^2 maps (f, φ, ψ , and the Jacobian operator) is C^1 . □

4.2 Riccati cache family

For each α , the steady-state Riccati equation with $A(\alpha), B(\alpha)$ and fixed Q, R, ρ gives:

$$\begin{aligned} P_\infty(\alpha) &= Q + \rho I + A(\alpha)^\top P_\infty(\alpha) A(\alpha) \\ &\quad - A(\alpha)^\top P_\infty(\alpha) B(\alpha) K_\infty(\alpha), \\ K_\infty(\alpha) &= (R + \rho I + B(\alpha)^\top P_\infty(\alpha) B(\alpha))^{-1} B(\alpha)^\top P_\infty(\alpha) A(\alpha), \\ C_\infty(\alpha) &= (R + \rho I + B(\alpha)^\top P_\infty(\alpha) B(\alpha))^{-1}. \end{aligned} \quad (12)$$

Proposition 4.2 (Smooth cache). *If $(A(\alpha), B(\alpha))$ is stabilisable for all α in a compact set $\mathcal{A} \subset \mathbb{R}^d$, then $\alpha \mapsto (P_\infty(\alpha), K_\infty(\alpha), C_\infty(\alpha))$ is C^1 on $\mathcal{A}$.*

4.3 The template

Definition 4.3 (Flat template). *A **flat template** for a differentially flat system with cost (Q, R, ρ) is a map*

$$\mathcal{T} : \mathbb{R}^d \rightarrow \mathbb{R}^{n \times n} \times \mathbb{R}^{m \times n} \times \mathbb{R}^{m \times m}, \quad \alpha \mapsto (P_\infty(\alpha), K_\infty(\alpha), C_\infty(\alpha)), \quad (13)$$

defined by the DARE (12).

Offline: evaluate $\mathcal{T}$ on a grid $\{\alpha_i\}_{i=1}^G \subset \mathcal{A}$ and store the cache triples.

Online: given current flat output derivatives, compute α_{now} , look up (or interpolate) the nearest cache, and run TinyMPC’s ADMM with $(A(\alpha_{\text{now}}), B(\alpha_{\text{now}}), K_{\infty}(\alpha_{\text{now}}), C_{\infty}(\alpha_{\text{now}}))$.

Online cost: $K \cdot O(n^2)$ (same as TinyMPC per iteration).

Remark 4.4 (Storage). *For a quadrotor ($n = 12, m = 4, d = 4$): each cache triple requires $12^2 + 4 \times 12 + 4^2 = 208$ floats = 832 bytes (float32). A 10^4 -point grid in $\mathbb{R}^4$ costs ≈ 8 MB—within microcontroller flash.*

5 Flat Template Algorithm

Input: Current state x_0 ;;

flat output derivatives $(\sigma, \dot{\sigma}, \dots)$ from planner or estimator;;

template $\mathcal{T}$;;

ADMM iterations K ;;

penalty $\rho > 0$;;

warm-start $(\bar{v}, \bar{\lambda})$.

Output: Control $u_0^* \in \mathbb{R}^m$.

Step 0: Flat parameter.;

$\alpha \leftarrow (\ddot{\sigma}_{1:3}, \psi)$; // quadrotor case

Step 1: Template lookup.;

$(A, B) \leftarrow (A(\alpha), B(\alpha))$; // evaluate or interpolate

$(P, K, C) \leftarrow \mathcal{T}(\alpha)$; // cached Riccati

Step 2: Reference.;

$x^* \leftarrow \varphi(\alpha)$;;

$u^* \leftarrow \psi(\alpha)$; // flat equilibrium

Shift coordinates: $\tilde{x}_0 \leftarrow x_0 - x^*$;;

update linear cost terms $q \leftarrow Q x^*, r \leftarrow R u^*$;

Step 3: ADMM (identical to TinyMPC).;

for $j = 1, \dots, K$ **do**

Primal: Forward rollout with cached K, C ;;

$\tilde{u}_k = -K \tilde{x}_k - C (B^T p_{k+1} + r + \rho(v_{u,k} - \lambda_{u,k}/\rho))$;;

$\tilde{x}_{k+1} = A \tilde{x}_k + B \tilde{u}_k$;

Slack: $v_{x,k} = \text{clip}(\tilde{x}_k + x^* + \lambda_{x,k}/\rho, x_{\min}, x_{\max}) - x^*$;

$v_{u,k} = \text{clip}(\tilde{u}_k + u^* + \lambda_{u,k}/\rho, u_{\min}, u_{\max}) - u^*$;

Dual: $\lambda_{x,k} += \rho(\tilde{x}_k - v_{x,k})$;;

$\lambda_{u,k} += \rho(\tilde{u}_k - v_{u,k})$;

end

return $u_0^* = \tilde{u}_0 + u^*$;

Algorithm 1: Flat Template MPC

6 Uniform Tunnel Guarantee

We connect the flat template to the tunnel theory [2].

6.1 Tunnel in original coordinates

Definition 6.1 (Tunnel radius). *For a fixed linearisation (A, B) at nominal $(\bar{x}, \bar{u})$, the **tunnel radius** is*

$$r(A, B) = \frac{2\alpha(A, B)}{L}, \quad (14)$$

where $\alpha(A, B) = 1 - \bar{\rho}(A - B K_\infty)$ is the spectral gap of the closed-loop operator (using $\bar{\rho}$ for spectral radius to avoid overloading with ADMM penalty ρ) and L is the Lipschitz constant of $\nabla^2 f$.

Assumption 6.2 (Uniform stabilisability). *There exists $\bar{\alpha} > 0$ such that for all $\alpha \in \mathcal{A}$:*

(i) $(A(\alpha), B(\alpha))$ is stabilisable,

(ii) $\alpha(A(\alpha), B(\alpha)) \geq \bar{\alpha} > 0$.

Theorem 6.3 (Uniform tunnel). *Under Assumption 6.2, the flat template MPC (Algorithm 1) has a **uniform** tunnel radius*

$$r_{\min} = \inf_{\alpha \in \mathcal{A}} r(\alpha) \geq \frac{2\bar{\alpha}}{L} > 0 \quad (15)$$

at every flat equilibrium in $\mathcal{A}$. The ADMM iterates contract at rate $\leq 1 - \bar{\alpha}$ uniformly:

$$\|w^{(j+1)} - w^*(\alpha)\| \leq (1 - \bar{\alpha}) \|w^{(j)} - w^*(\alpha)\| \quad (16)$$

for all $\alpha \in \mathcal{A}$ and all $w^{(0)}$ with $\|w^{(0)} - w^*(\alpha)\| < r_{\min}$, where $w = (x, u, v, \lambda)$ is the full primal-dual variable.

Corollary 6.4 (Full flat-reachable envelope). *The union of all α -tunnels covers a neighbourhood of the entire flat equilibrium manifold. For a quadrotor with $\mathcal{A}$ spanning accelerations up to $a_{\max}$ and all yaw angles, this includes all attitudes reachable by $\|a\| \leq a_{\max}$ —far beyond hover.*

7 Variant 3: Analytic Template (Zero-Iteration MPC)

The flat template (§4) eliminates the fixed-nominal limitation but still runs K ADMM iterations online. We now eliminate the iterations entirely.

7.1 Key observation

When box constraints are inactive, the ADMM solution after $K \rightarrow \infty$ iterations reduces to the unconstrained LQR:

$$u_0^* = u^*(\alpha) - K_\infty(\alpha) (x_0 - x^*(\alpha)). \quad (17)$$

This is a **single matrix–vector product**—no iteration, no slack update, no dual update.

7.2 Quadrotor: explicit $A(\alpha), B(\alpha)$

For a quadrotor with state $x = (p, \phi, \theta, \psi, v, \omega)^\top \in \mathbb{R}^{12}$ and input $u = (u_1, u_2, u_3, u_4)^\top \in \mathbb{R}^4$ (individual motor thrusts), the flat parameter is $\alpha = (a_x, a_y, a_z, \psi)$.

The equilibrium attitude follows from the thrust direction:

$$T^* = m\|a + g e_3\|, \quad \phi^* = \arcsin\left(\frac{m(a_x \sin \psi - a_y \cos \psi)}{T^*}\right), \quad \theta^* = \arctan\left(\frac{a_x \cos \psi + a_y \sin \psi}{a_z + g}\right). \quad (18)$$

The linearised dynamics at this equilibrium have the block structure:

$$A_c(\alpha) = \begin{pmatrix} 0 & 0 & I_3 & 0 \\ 0 & 0 & 0 & W(\phi^*, \theta^*) \\ 0 & G(\alpha) & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad B_c(\alpha) = \begin{pmatrix} 0 \\ 0 \\ b(\alpha) \cdot \mathbf{1}^\top \\ J^{-1} M_{\text{mix}} \end{pmatrix}, \quad (19)$$

where $G(\alpha) \in \mathbb{R}^{3 \times 3}$ encodes the gravity coupling, $W(\phi^*, \theta^*)$ is the Euler kinematic map, $b(\alpha) = k_t \cdot e_3^B(\alpha)/m$, and $J^{-1} M_{\text{mix}}$ is the torque allocation. Discretization: $A_d = I + A_c \Delta t$, $B_d = B_c \Delta t$.

7.3 Perturbation expansion of K_∞

Write $\alpha = \alpha_0 + \delta\alpha$ where $\alpha_0 = (0, 0, 0, 0)$ is hover. The DARE perturbation theory gives:

$$P_\infty(\alpha) = P_0 + \delta P + O(\|\delta\alpha\|^2), \quad (20)$$

where δP solves the discrete Lyapunov equation

$$\delta P = A_{\text{cl},0}^\top \delta P A_{\text{cl},0} + S(\delta A, \delta B, P_0, K_0), \quad (21)$$

with $A_{\text{cl},0} = A_0 - B_0 K_0$.

Proposition 7.1 (Analytic gain). *The first-order analytic gain is*

$$K(\alpha) = K_0 + \sum_{i=1}^d \delta\alpha_i K_i + O(\|\delta\alpha\|^2), \quad (22)$$

where each $K_i \in \mathbb{R}^{m \times n}$ is a constant matrix given by

$$K_i = C_0 (\delta B_i^\top P_0 A_0 + B_0^\top P_i A_0 + B_0^\top P_0 \delta A_i - \delta B_i^\top P_0 B_0 K_0 - B_0^\top P_i B_0 K_0), \quad (23)$$

with $C_0 = (R + B_0^\top P_0 B_0)^{-1}$. All quantities are constant.

7.4 The zero-iteration control law

Combining (17) and (22):

$$u = u^*(\alpha) - \left(K_0 + \sum_{i=1}^d \delta\alpha_i K_i \right) (x_0 - x^*(\alpha)). \quad (24)$$

Online cost: $O(dmn)$. For a quadrotor: $O(4 \cdot 4 \cdot 12) = O(192)$ multiply-accumulate operations. With equilibrium computation: ≈ 260 total ops.

Remark 7.2 (Constraints). *When constraints are active, clamp the output: $u = \text{clip}(u^*(\alpha) - K(\alpha)(x_0 - x^*(\alpha)), u_{\min}, u_{\max})$. This is a zeroth-order ADMM approximation.*

8 Variant 4: Barrier-Augmented Template

The Analytic Template handles constraints only by clipping—the gain $K(\alpha)$ is unaware of limits. We now show that log-barrier functions can be absorbed into the DARE cache, yielding a zero-iteration controller that is inherently constraint-aware.

8.1 Interior-point formulation

For box constraints $u_{\min} \leq u \leq u_{\max}$, the log-barrier method replaces these with a smooth penalty:

$$\mathcal{B}_\mu(u) = -\mu \sum_{j=1}^m [\log(u_j - u_{\min,j}) + \log(u_{\max,j} - u_j)], \quad (25)$$

where $\mu > 0$ is the barrier parameter.

8.2 Input barrier Hessian

At the flat equilibrium $u^*(\alpha)$, the barrier Hessian is a known diagonal matrix:

$$H_\mu(\alpha) = \mu \cdot \text{diag} \left(\frac{1}{(u_j^* - u_{\min,j})^2} + \frac{1}{(u_{\max,j} - u_j^*)^2} \right). \quad (26)$$

Large when $u^*(\alpha)$ is near a bound; small when far from all bounds.

8.3 State barrier Hessian

For state constraints $x_{\min} \leq x \leq x_{\max}$ (e.g., position bounds $|p_i| \leq L$):

$$H_{x,\mu}(\alpha) = \mu \cdot \text{diag} \left(\frac{1}{(x_i^* - x_{\min,i})^2} + \frac{1}{(x_{\max,i} - x_i^*)^2} \right), \quad (27)$$

applied only to constrained state dimensions (unconstrained dimensions contribute zero). In error coordinates ($x_{\text{pos}}^* = 0$), this depends on the reference-to-boundary distance.

8.4 Barrier-augmented DARE

Define barrier-augmented costs:

$$R_\mu(\alpha) = R + H_\mu(\alpha), \quad (28)$$

$$Q_\mu(\alpha) = Q_{\text{lqr}} + H_{x,\mu}(\alpha), \quad (29)$$

where Q_{lqr} is the terminal cost from standard discrete LQR.

Solving the augmented DARE:

$$P_\mu = Q_{\mu,\rho} + A_d^\top P_\mu (A_d - B_d G_\mu^{-1} B_d^\top P_\mu A_d), \quad (30)$$

where $Q_{\mu,\rho} = Q_\mu + \rho I$ and $G_\mu = R_\mu + \rho I + B_d^\top P_\mu B_d$, yields the barrier-augmented gain $K_\mu(\alpha) = G_\mu^{-1} B_d^\top P_\mu A_d$.

The zero-iteration barrier control law:

$$u = u^*(\alpha) - \left(K_{\mu,0} + \sum_{i=1}^d \delta \alpha_i \frac{\partial K_\mu}{\partial \alpha_i} \right) (x_0 - x^*(\alpha)). \quad (31)$$

Online cost: ~ 260 ops (identical to Analytic Template).

Storage: $5 \times 4 \times 12 = 240$ floats (identical).

8.5 α -dependent conservatism

At hover, the four motors produce equal thrust at midrange and H_μ is moderate. At aggressive maneuvers (large $\|a\|$), some motors approach saturation and H_μ grows in those channels, automatically increasing conservatism. The scheduling variable α simultaneously parameterizes the linearization, the equilibrium, *and* the constraint tightness.

8.6 Optional: IPM refinement

For tighter constraint satisfaction, K_{IPM} Newton steps can be run online:

1. Start from the zero-iteration output $u^{(0)}$.
2. Recompute H_μ at current $u^{(k-1)}$ (not at equilibrium).
3. Solve one Riccati backward pass with updated $R_\mu \rightarrow \delta u$.
4. $u^{(k)} = u^{(k-1)} + \eta \delta u$ (Newton step).

Each step costs $O(n^2)$ (same as one ADMM iteration). Unlike ADMM (linear convergence), Newton–IPM converges *quadratically*: 2–3 iterations typically suffice.

8.7 Limitation: reference outside feasible region

The barrier Hessian penalizes error-state excursions from the reference. If the reference itself exceeds the constraint boundary (e.g., a circular trajectory outside a square bound), no error-state cost modification can prevent violation. Only the ADMM projection (which maps absolute bounds to per-step error-state bounds) can handle this case.

Key result: In the circle-in-square experiment, Barrier = Analytic = 88.4 mm max violation, because $R - L = 88$ mm is the geometric overshoot of the reference.

9 Barrier-Augmented Algorithm

Input: Constraint bounds $u_{\min}, u_{\max}, x_{\min}, x_{\max}$ (optional), barrier μ , penalty ρ .

Output: $K_{\mu,0}, \{\partial K_{\mu}/\partial \alpha_i\}_{i=1}^4, \alpha_0$.

Offline:

```

 $\alpha_0 \leftarrow (0, 0, 0, 0)$  ; // hover
 $(x^*, u^*) \leftarrow \Phi(\alpha_0)$  ; // flat equilibrium
 $H_{\mu} \leftarrow \text{Eq. (26) from } u^* \text{ and input bounds}$ 
 $H_{x,\mu} \leftarrow \text{Eq. (27) from } x^* \text{ and state bounds}$ 
 $R_{\mu} \leftarrow R + H_{\mu}; \quad Q_{\mu} \leftarrow Q_{\text{lqr}} + H_{x,\mu}$ 
Solve DARE (30) with  $R_{\mu}, Q_{\mu} \rightarrow K_{\mu,0}$ 
for  $i = 1, \dots, 4$  do
     $K_{\mu}^+ \leftarrow \text{solve barrier DARE at } \alpha_0 + \varepsilon e_i$ 
     $K_{\mu}^- \leftarrow \text{solve barrier DARE at } \alpha_0 - \varepsilon e_i$ 
     $\partial K_{\mu}/\partial \alpha_i \leftarrow (K_{\mu}^+ - K_{\mu}^-)/(2\varepsilon)$  ; // central diff
end
Store  $(K_{\mu,0}, \partial_1 K_{\mu}, \dots, \partial_4 K_{\mu}, \alpha_0)$ : 240 floats.

```

Online (per control step):

```

 $\alpha \leftarrow \text{current flat parameter}$ 
 $\delta \alpha \leftarrow \alpha - \alpha_0$ 
 $(x^*, u^*) \leftarrow \Phi(\alpha)$ 
 $K_{\mu} \leftarrow K_{\mu,0} + \sum_i \delta \alpha_i \partial_i K_{\mu}$  ; // 192 MACs
 $u \leftarrow u^* - K_{\mu}(x - x^*)$  ; // 48 MACs
 $u \leftarrow \text{clip}(u, u_{\min}, u_{\max})$  ; // 4 comparisons

```

Algorithm 2: Barrier-Augmented Template MPC

10 Foolproof Implementation

Everything above reduces to the following. No optimisation theory is needed to implement it.

10.1 Constants (your quadrotor)

Fix these once for your hardware:

Symbol	Meaning	How to get it
m	mass (kg)	weigh it
g	gravity (9.81)	constant
J_x, J_y, J_z	inertia (kg m ²)	CAD or swing test
k_t	per-motor thrust coeff	motor datasheet
k_m	per-motor torque coeff	motor datasheet
ℓ	arm length (m)	measure it
$Q \in \mathbb{R}^{12 \times 12}$	state cost	diagonal, you choose
$R \in \mathbb{R}^{4 \times 4}$	input cost	diagonal, you choose
Δt	control period (s)	your timer
$u_{\min}, u_{\max}$	actuator limits	motor datasheet

10.2 Offline (run once, on your laptop)

Input: $m, g, J, k_t, k_m, \ell, Q, R, \Delta t$;

(optional) $\mu, u_{\min}, u_{\max}, x_{\min}, x_{\max}$.

Output: $K_0 \in \mathbb{R}^{4 \times 12}$;

$K_1, K_2, K_3, K_4 \in \mathbb{R}^{4 \times 12}$;

$\alpha_0 \in \mathbb{R}^4$.

1. Hover equilibrium.

$\alpha_0 \leftarrow (0, 0, 0, 0)$; // at hover, $a = (0, 0, g)$ but code uses $(0, 0, 0, 0)$

$u_0^* \leftarrow (T^*/(4k_t)) \cdot \mathbf{1} \in \mathbb{R}^4$ where $T^* = mg$

2. Hover linearisation.

$(A_0, B_0) \leftarrow \text{linearise}(\alpha_0)$;

// Algorithm 4

3. Solve DARE at hover.

if *Barrier variant* ($\mu > 0$) **then**

$H_\mu \leftarrow \text{Eq. (26) from } (u_0^*, u_{\min}, u_{\max})$
 $H_{x,\mu} \leftarrow \text{Eq. (27) from } (x_0^*, x_{\min}, x_{\max})$
 $R_\mu \leftarrow R + H_\mu$;
 $Q_\mu \leftarrow \text{dlqr}(A_0, B_0, Q, R_\mu) + H_{x,\mu}$
 $P_0 \leftarrow \text{dare}(A_0, B_0, Q_\mu + \rho I, R_\mu + \rho I)$

end

$P_{\text{lqr}} \leftarrow \text{dlqr}(A_0, B_0, Q, R)$

$P_0 \leftarrow \text{dare}(A_0, B_0, P_{\text{lqr}} + \rho I, R + \rho I)$

$K_0 \leftarrow (R_{\text{aug}} + B_0^\top P_0 B_0)^{-1} B_0^\top P_0 A_0$

4. Central differences (4 directions).

$\varepsilon \leftarrow 10^{-3}$

for $i = 1, \dots, 4$ **do**

$\alpha^+ \leftarrow \alpha_0 + \varepsilon e_i, \quad \alpha^- \leftarrow \alpha_0 - \varepsilon e_i$
 $(A^\pm, B^\pm) \leftarrow \text{linearise}(\alpha^\pm)$
 $K^\pm \leftarrow \text{solve DARE (same procedure as Step 3) at } \alpha^\pm$
 $K_i \leftarrow (K^+ - K^-)/(2\varepsilon)$;

// central difference

end

5. Store.

Save $(K_0, K_1, K_2, K_3, K_4, \alpha_0)$: $5 \times 4 \times 12 = 240$ numbers.

Algorithm 3: Offline Precomputation (Analytic or Barrier)

10.3 linearise(α)

Input: $\alpha = (a_x, a_y, a_z, \psi),;$
quadrotor constants $m, g, k_t, k_m, \ell, J, \Delta t$.
Output: $(A_d, B_d) \in \mathbb{R}^{12 \times 12} \times \mathbb{R}^{12 \times 4}$.

```

// Equilibrium attitude from flat output
 $T^* \leftarrow m \sqrt{a_x^2 + a_y^2 + (a_z + g)^2}$ 
 $\phi^* \leftarrow \arcsin(m (a_x \sin \psi - a_y \cos \psi) / T^*)$ 
 $\theta^* \leftarrow \arctan((a_x \cos \psi + a_y \sin \psi) / (a_z + g))$ 

// Gravity coupling matrix  $G \in \mathbb{R}^{3 \times 3}$ 
 $(s_\phi, c_\phi, s_\theta, c_\theta, s_\psi, c_\psi) \leftarrow \text{trig of } (\phi^*, \theta^*, \psi)$ 
 $G[0, :] \leftarrow T^*/m \cdot (-c_\psi s_\theta s_\phi + s_\psi c_\phi, c_\psi c_\theta c_\phi, -s_\psi s_\theta c_\phi + c_\psi s_\phi)$ 
 $G[1, :] \leftarrow T^*/m \cdot (-s_\psi s_\theta s_\phi - c_\psi c_\phi, s_\psi c_\theta c_\phi, c_\psi s_\theta c_\phi + s_\psi s_\phi)$ 
 $G[2, :] \leftarrow T^*/m \cdot (-c_\theta s_\phi, -s_\theta c_\phi, 0)$ 

// Assemble  $A_c, B_c$  (continuous time)
 $A_c[0:3, 6:9] \leftarrow I_3;;$ 
 $A_c[3:6, 9:12] \leftarrow W(\phi^*, \theta^*);;$ 
 $A_c[6:9, 3:6] \leftarrow G$ 
 $B_c[6:9, :] \leftarrow k_t/m \cdot e_3^B \cdot \mathbf{1}^\top;;$ 
 $B_c[9:12, :] \leftarrow J^{-1} M_{\text{mix}}$ 

// Discretise (forward Euler)
 $A_d \leftarrow I_{12} + A_c \Delta t;;$ 
 $B_d \leftarrow B_c \Delta t$ 
return  $(A_d, B_d)$ 

```

Algorithm 4: linearise(α): linearisation at flat parameter α

10.4 Online (every control cycle)

Input: Stored constants $K_0, K_1, K_2, K_3, K_4 \in \mathbb{R}^{4 \times 12}$;
 $\alpha_0 = (0, 0, 0, 0)$;
 $u_{\min}, u_{\max} \in \mathbb{R}^4$;
mass m , gravity g , thrust coeff k_t .

// Run this at every time step

while true do

A. Read sensors.
 $x \leftarrow \text{IMU} + \text{estimator output} = (p_x, p_y, p_z, \phi, \theta, \psi, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)^T \in \mathbb{R}^{12}$

B. Get flat parameter.
 $(a_x, a_y, a_z, \psi_{\text{ref}}) \leftarrow \text{trajectory planner output}$; // commanded accel + yaw
 $\alpha \leftarrow (a_x, a_y, a_z, \psi_{\text{ref}})$
 $\delta\alpha \leftarrow \alpha - \alpha_0$; // 4 subtractions

C. Flat equilibrium.
 $T^* \leftarrow m \sqrt{a_x^2 + a_y^2 + (a_z + g)^2}$
 $\phi^* \leftarrow \arcsin(m(a_x \sin \psi_{\text{ref}} - a_y \cos \psi_{\text{ref}})/T^*)$
 $\theta^* \leftarrow \arctan((a_x \cos \psi_{\text{ref}} + a_y \sin \psi_{\text{ref}})/(a_z + g))$
 $x^* \leftarrow (p^{\text{ref}}, \phi^*, \theta^*, \psi_{\text{ref}}, v^{\text{ref}}, 0, 0, 0)^T$
 $u^* \leftarrow (T^*/(4k_t)) \cdot \mathbf{1}$

D. Form gain (4 scalar-matrix multiplies + 4 adds).
 $K \leftarrow K_0$; // copy $4 \times 12 = 48$ numbers
for $i = 1, \dots, 4$ **do**
 | $K \leftarrow K + \delta\alpha_i \cdot K_i$; // 48 MACs each, 192 total
end

E. Compute control (one 4×12 matvec).
 $\tilde{x} \leftarrow x - x^*$; // 12 subtractions
 $\tilde{u} \leftarrow K \cdot \tilde{x}$; // 48 MACs
 $u \leftarrow u^* - \tilde{u}$; // 4 subtractions

F. Clamp.
for $j = 1, \dots, 4$ **do**
 | $u_j \leftarrow \max(u_{\min, j}, \min(u_{\max, j}, u_j))$
end

G. Send.
Write u to motor controller.
Wait for next timer tick.

end

Algorithm 5: Online Control Loop (the whole thing)

Remark 10.1 (Operation count). *Steps A, B: sensor read + 4 subtractions. Step C: 1 sqrt, 1 asin, 1 atan, 4 trig, 6 multiplies (≈ 20 ops on CORDIC, ≈ 50 on FPU). Step D: 192 MACs. Step E: 48 MACs + 16 adds. Step F: 4 comparisons. Total: ≈ 260 arithmetic operations per control cycle. At 1 GFLOP/s (low-end MCU): $0.26 \mu\text{s}$. At 200 MHz FPGA with 48 parallel MACs: $\sim 75 \text{ ns}$.*

11 FPGA Architecture

The zero-iteration law (24) maps to hardware with no control flow: **pure datapath**.

11.1 Pipeline stages

Stage	Operation	Hardware	Latency
1	$\alpha \leftarrow (\ddot{\sigma}, \psi)$	register read	1 clk
2	$\delta\alpha \leftarrow \alpha - \alpha_0$	d adders	1 clk
3	$x^*(\alpha), u^*(\alpha)$	CORDIC (trig)	~ 10 clk
4	$\tilde{x} \leftarrow x_0 - x^*$	n adders	1 clk
5	$K(\alpha) = K_0 + \sum_i \delta\alpha_i K_i$	$d \cdot mn$ MACs	$\sim mn/P$ clk
6	$\tilde{u} = K(\alpha) \tilde{x}$	mn MACs	$\sim mn/P$ clk
7	$u = u^* - \tilde{u}$; clip	m add + compare	1 clk

P = number of parallel MAC units. With $P = 48$ (one MAC per element of K): stages 5–6 each take 1 clock cycle. Total latency: ~ 15 clock cycles. At 200 MHz: ~ 75 ns.

11.2 Resource estimate (quadrotor)

Resource	Count	Note
Constant matrices	$1 + d = 5$	$K_0, K_1, \dots, K_4 \in \mathbb{R}^{4 \times 12}$
Stored coefficients	$5 \times 48 = 240$	float16 or fixed-point
Storage	480 bytes	fits in BRAM
DSP slices	≤ 48	one per K entry
CORDIC cores	3	sin, cos, arctan

Fits on a low-end FPGA (e.g. Xilinx Artix-7, Lattice iCE40). No soft-core CPU, no OS, no memory allocator. The controller is a circuit.

12 Complete Hierarchy

The five variants form a complexity cascade:

Online Relin $\xrightarrow{\text{cache}}$ Flat Template $\xrightarrow{K \rightarrow 0}$ Barrier Templ. $\xrightarrow{\mu \rightarrow 0}$ Analytic Templ. $\xrightarrow{\text{1st order}}$ Gain Sched.

Variant	Linearisation	Iters	Online cost	Constraints	Valid region
Online Relin	fresh Jacobian	K	$O(n^3 + Kn^2)$	Hard	global
Flat Template	α -grid cache	K	$K \cdot O(n^2)$	Hard	flat envelope
Barrier Templ.	hover + ∂K_μ	0	$O(dmn)$	Soft	flat envelope
Analytic Templ.	hover + ∂K	0	$O(dmn)$	Clip	flat envelope
Gain Schedule	hover (fixed)	0	$O(dmn)$	Clip	near hover

All variants with α -parameterization share the uniform tunnel guarantee (Theorem 6.3). The key insight: flatness compresses the scheduling space from $\mathbb{R}^{12}$ to $\mathbb{R}^4$ via $\alpha = (a_x, a_y, a_z, \psi)$, so only 4 sensitivity matrices (240 floats) are needed.

13 Experiment Results

All simulations use a Crazyflie 2.1 model (mass 35 g, arm 46 mm) at 50 Hz. Python reference implementation; wall times are for relative comparison only.

13.1 Hover stabilization

Initial offset (0.2, 0.2, -0.2) m. $N = 10$, $\rho = 85$, 100 steps.

	TinyMPC	Online Relin	Flat Templ.	Analytic
Avg. pos. err (m)	0.064	0.095	0.138	0.145
Avg. ADMM iters	80.0	16.7	7.8	0
Wall time (s)	2.85	12.13	0.44	0.13

At hover, TinyMPC’s fixed linearization is exact (α_0 is hover) and achieves the best error. But the Analytic Template is 22 $\times$ faster.

13.2 Figure-eight tracking

xz -plane, amplitude 0.5 m, period 3.7 s. $N = 15$, $\rho = 5$, 200 steps.

	TinyMPC	Online Relin	Flat Templ.	Analytic
Avg. pos. err (m)	0.033	0.035	0.035	0.036
Avg. ADMM iters	8.0	5.1	3.6	0
Wall time (s)	1.16	22.02	0.75	0.26

All four variants achieve comparable accuracy. The Flat Template is 1.5 $\times$ faster than TinyMPC; the Analytic Template is 4.5 $\times$ faster. Online Relin is 29 $\times$ slower due to per-step DARE solves.

13.3 Circle-in-square (constrained tracking)

Circular trajectory ($R = 0.3$ m) in the xy -plane, inscribed square $|x|, |y| \leq L = R/\sqrt{2}$. $N = 20$, $\rho = 50$, 500 steps.

	Online Relin (ADMM)	Flat Templ. (ADMM)	Barrier (zero-iter)	Analytic (zero-iter)
Avg. pos. err (m)	0.062	0.062	0.017	0.017
Max viol. (mm)	0.0	0.0	88.4	88.4
Mean viol. (mm)	0.0	0.0	45.8	45.7
Avg. ADMM iters	450.8	450.8	0	0
Wall time (s)	168.1	157.4	0.65	0.62

Key observations:

- ADMM variants achieve 0 mm violation by encoding absolute constraints as per-step error-state bounds.
- Zero-iteration variants show 88.4 mm violation = $R - L$ (geometric limit).
- Barrier = Analytic because the barrier Hessian cannot override a reference outside the feasible region.
- $240\times$ wall-time speedup from ADMM to zero-iteration.

14 Python API Reference

14.1 Source modules

Module	Description
src/quadrotor.py	13-state quaternion dynamics (autograd)
src/tinympc.py	Core ADMM-based MPC solver
src/flat_linearization.py	α -parameterized analytic Jacobians
src/flat_template.py	Flat Template: DARE cache grid + ADMM
src/analytic_template.py	Analytic Template: $K_0 + \sum \delta\alpha_i K_i$
src/barrier_template.py	Barrier Template: log-barrier in DARE cache
src/rho_adapter.py	ADMM penalty ρ adaptation

14.2 Flat linearization API

Listing 1: flat_linearization.py

```
1 from src.flat_linearization import (
2     get_quad_params, # QuadrotorDynamics -> dict
3     flat_equilibrium, # alpha -> (x_eq, u_eq)
4     linearise, # alpha -> (A_d, B_d)
5     quat_to_euler_state, # 13D quat -> 12D Euler
6     euler_to_quat, # (phi, theta, psi) -> quat
7 )
8
9 # Setup
10 quad = QuadrotorDynamics(...)
11 params = get_quad_params(quad)
12
13 # Flat parameter: [a_x, a_y, a_z, psi]
14 alpha = np.array([0.0, 0.0, 0.0, 0.0]) # hover
15
16 # Equilibrium state (12D) and input (4D motor thrusts)
17 x_eq, u_eq = flat_equilibrium(alpha, params)
18
19 # Analytic Jacobians (no autograd)
20 A_d, B_d = linearise(alpha, params) # (12x12), (12x4)
```

14.3 Analytic Template API

Listing 2: analytic_template.py

```
1 from src.analytic_template import AnalyticTemplate
2
3 ctrl = AnalyticTemplate(params, Q, R, rho=5.0)
4 ctrl.precompute() # solves 9 DAREs, stores 240 floats
5
6 # Online: ~260 ops per call
7 alpha = np.array([a_x, a_y, a_z, psi])
8 u = ctrl.control(x, alpha, u_min=u_lo, u_max=u_hi)
```

14.4 Barrier Template API

Listing 3: barrier_template.py

```
1 from src.barrier_template import BarrierTemplate
2
3 ctrl = BarrierTemplate(
4     params, Q, R, rho=50.0, mu=1e-3,
5     u_min=u_lo, u_max=u_hi, # input constraints
6     x_min=x_lo, x_max=x_hi, # state constraints
7 )
8 ctrl.precompute() # barrier-augmented DAREs
9
10 # Online: ~260 ops (identical datapath)
11 u = ctrl.control(x, alpha, u_min=u_lo, u_max=u_hi)
```

14.5 Flat Template API

Listing 4: flat_template.py

```
1 from src.flat_template import FlatTemplateMPC
2
3 # Grid of alpha values for cache
4 grid = [np.array([0, 0, 0, 0]), ...]
5
6 ctrl = FlatTemplateMPC(params, Q, R, N=15, rho=5.0,
7                       grid_alphas=grid)
8 ctrl.set_bounds(umin=u_lo, umax=u_hi,
9                xmin=x_lo, xmax=x_hi)
10 ctrl.set_tols_iters(max_iter=10,
11                    abs_pri_tol=1e-3, abs_dua_tol=1e-3)
12
13 # Online: ADMM solve with cached matrices
14 x_traj, u_traj, status, iters = ctrl.solve(
15     x0_error, alpha
16 )
```

14.6 TinyMPC API

Listing 5: tinympc.py

```
1 from src.tinympc import TinyMPC
2
3 ctrl = TinyMPC(A_hover, B_hover, Q, R,
4               Nsteps=15, rho=5.0)
5 ctrl.set_bounds(umin=u_lo, umax=u_hi,
6               xmin=x_lo, xmax=x_hi)
7
8 # Online: ADMM with fixed (A, B)
9 x_traj, u_traj, status, iters = ctrl.solve_admm(
10     x_init, u_init, x_ref=x_ref, u_ref=u_ref
11 )
```

15 Discussion

15.1 Connection to TTT

The flat template is the LQ envelope of [2] parameterised by the flat coordinate α . The envelope moves with the operating point; flatness provides the global chart in which this motion is smooth. The tunnel theorem of [2] gives the contraction guarantee at each α ; Theorem 6.3 is the uniform version over all α . The analytic template (Proposition 7.1) is the first-order Taylor expansion of the envelope in α —the tunnel linearised in its own parameter space.

15.2 Why Barrier Cannot Enforce Absolute Constraints

The barrier Hessian penalizes deviations from the *reference* in error coordinates. When the reference itself violates the constraint (e.g., a circular trajectory outside a square bound), no error-state cost modification can prevent violation. The costate ∇V at the reference does not capture the directional constraint information needed near boundaries. Only ADMM’s iterative projection can map absolute bounds to per-step error-state bounds—the “missing ingredient” that zero-iteration controllers lack.

15.3 Gain scheduling, done right

Classical gain scheduling computes $K(\alpha)$ by interpolating between hand-tuned operating points. The analytic template produces the same object but:

1. The schedule variable α is *derived* (from flatness), not chosen by intuition.
2. The gain perturbation K_i is *computed* (from the DARE), not tuned.
3. The valid region is *guaranteed* (by the tunnel), not assumed.

15.4 Higher-order templates

The first-order expansion (22) is accurate near hover. For extreme manoeuvres, extend to second order: $K(\alpha) = K_0 + \sum_i \delta\alpha_i K_i + \sum_{i,j} \delta\alpha_i \delta\alpha_j K_{ij} + \dots$ Each K_{ij} is a constant matrix. Online cost scales as $O(d^2 mn)$ —still $O(1)$ iterations, just more MACs. On FPGA, second order adds $\binom{d+1}{2} = 10$ additional scalar–matrix multiplies for $d = 4$.

15.5 Beyond quadrotors

Any differentially flat system admits an analytic template:

- **Fixed-wing aircraft:** $\sigma = (x, y, z, \psi)$, $d = 4$.
- **Planar bipedal robot:** flat output = CoM + swing foot.
- **Differentially flat manipulators:** $\sigma =$ joint positions (trivially flat), $d = m$. Degenerates to computed-torque controller.

15.6 What this does not do

The analytic template addresses model mismatch from operating-point dependence. It does *not* address:

- Unmodelled dynamics (wind, flex, latency) — use a disturbance observer or L1 adaptive.
- Non-flat systems — no flat coordinate chart exists; use the grid-based template.
- Hard constraint satisfaction under sustained saturation — fall back to ADMM.

References

- [1] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester. TinyMPC: Model-predictive control on resource-constrained microcontrollers. *arXiv:2310.16985*, 2023.
- [2] J. Zhang. The tunnel theory. 2026.
- [3] P. Lancaster and L. Rodman. *Algebraic Riccati Equations*. Oxford University Press, 1995.
- [4] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [5] A. Jordana, S. Kleff, A. Meduri, J. Carpentier, N. Mansard, and L. Righetti. Stagewise Newton method for dynamic game control problems. *arXiv:2307.09298*, 2023.
- [6] W. Rugh and J. Shamma. Research on gain scheduling. *Automatica*, 36(10):1401–1425, 2000.
- [7] G. Hewer. An iterative technique for the computation of the steady-state gains for the discrete optimal regulator. *IEEE Trans. Automat. Control*, 16(4):382–384, 1971.
- [8] M. Konstantinov, P. Petkov, and N. Christov. Perturbation analysis of the discrete Riccati equation. *Kybernetika*, 29(1):18–29, 1993.